a) Paso a paso de la implementación: Orden en el que habéis ido desarrollando las fases.

1º Implementación en el sitio web de los artículos para comprar por categorías,tallas... y por consiguiente la creación y su reflejo en la base de datos (tienda_ropa) y de sus tablas estáticas correspondientes:
a) productos
b) producto_tallas
c) tallas

2º Implementación de la cabecera de navegación de usuario no logeado (headerhome.php)



y de la cabecera de navegación de usuario logeado (headerregistrado.php).



*Se han implementado según las cookies:

-Usuario logeado: una cookie
-Usuario no logeado: otra cookie.

Que se determinan por el navegador.

3º Implementación del registro de un usuario cumpliendo " **Fase 2: Autenticación (Registro) "**, que consiste en la creación automática de la tabla *usuarios* según los datos de registro del cliente introducidos en el formulario correspondiente- que tiene validaciones por html y javascript y se cumple el punto **" Fase 1: Ampliación de la Base de Datos**, tabla *usuarios".*

4º Implementación del login (cabecera "Mi cuenta") cumpliendo la " **Fase 2: Autenticación (Login), Requisito de Seguridad: Las contraseñas jamás se guardan en texto plano "**, que consiste en la validación contra la tabla *usuarios* y en concreto contra la columna *usuario* y el hash de contraseña. (se anexa un ejemplar de usuario en la tabla, cumpliendo el punto **"2. Export de la Base de Datos: y datos de prueba (usuarios creados, direcciones y pedidos ficticios) "** Ademas, se cumple dentro de la **"fase 2: Al hacer login con éxito, el sistema**

debe iniciar una sesión ($_SESSION) guardando el ID del usuario" y por último se cumple el punto **"Fase 1: Ampliación de la Base de Datos".**

5º Una vez realizado el registro y login, se ha procedido a probar sendas cabeceras descritas en el punto 2º.

6º El usuario logeado ya puede o no, rellenar la tabla *direcciones* **de la Fase 1** a través de un formulario cumpliendo así, **la fase 3, Mis direcciones** en su totalidad. (En la cabecera de headerregistrado.php se rellenarían), pudiendo modificarse cada una de las registradas.

7º: Una vez implementados los puntos 3º y 4, se ha procedido a completar el código para que el usuario logeado pueda desconectarse y así cumplir el apartado del logout **de la Fase 2**.

(Los apartados 6º y 7º pueden alternarse, no es necesario cumplir ese orden)

8º Para cumplir la **Fase 4: Inteligencia en el Checkout, si el usuario está Logueado,** si no tiene ninguna dirección de entrega registrada el formulario mencionado en el punto 6º de direcciones, le solicita rellenar una dirección de Entrega o Facturación, pero hasta que no tenga una dirección de Entrega, no se autorellenarán los campos del formulario ni se le "enviara" el pedido a dicha dirección de entrega.

Por el contrario, si ya tiene una dirección de Entrega previamente registrada, solo le dejará registrar tantas direcciones de Facturación como el usuario quiera, sin límites. *(La práctica no indica un máximo de cuantas de Facturación hay que registrar).

*El usuario logeado tiene posibilidad de "arrepentirse" del pedido cancelando la compra en el momento correspondiente.

9º Para cumplir la **Fase 4: Inteligencia en el Checkout, Si el usuario es "Invitado" (No logueado) ,** si pincha en *"¿Ya tienes cuenta? Inicia sesión para ir más rápido"* del formulario, le lleva al formulario de Mi cuenta= inicio sesión accediendo entonces ya a las funciones y funcionalidades mencionadas anteriormente del usuario logeado.

*El usuario no registrado tiene posibilidad de “arrepentirse” del pedido cancelando la compra en el momento correspondiente.

10º Por último, para realizar la Fase 3: El panel "Mi Cuenta", Mis pedidos, se pueden ver las opciones del mismo, haciendo click en el carrito de cualquier header antes mencionados.

Como consecuencia, las tablas relacionadas con los pedidos, se actualizarán de manera dinámica tanto si es una precompra=antes de “pagar”, como compra “ya pagada” o “pedido cancelado”.

*Finalizando con este último apartado, toda la práctica.

b) Decisiones técnicas: ¿Cómo habéis estructurado los archivos? ¿Por qué habéis usado ciertas consultas SQL o funciones de PHP? Explicad el "cómo" y el "por qué".

-Los archivos se han estructurado como si fueran bloques, funciones o recursos independientes para poder aislar los posibles fallos e identificar funciones (aunque en algún caso inevitable, como en los controllers, dependan de otros archivos).

- Se han usado consultas SQL para crear de manera más rapida toda las tablas en un solo comando o modificarlas.

*Se anexan archivos .odt con información adicional sobre ello.

- Se han usado funciones PHP sobre todo para no ejecutar javascript y familiarizarse mas con la codificación PHP, minimizando al máximo el uso de dicho .js y planteando a la IA mis propias implementaciones.

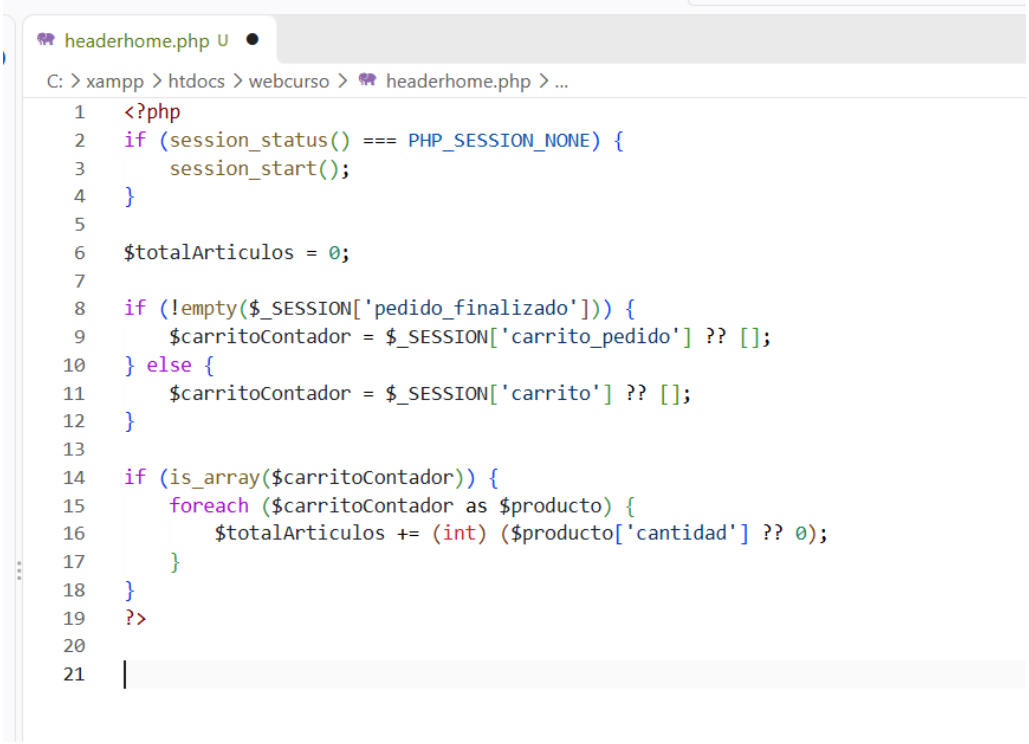
*También se anexan archivos .odt con información adicional sobre ello.

c) 4/10 - Memoria Técnica y Documentación: Explicación clara del paso a paso, justificación de las decisiones técnicas tomadas, inclusión de capturas de pantalla del desarrollo y transparencia en el uso de la IA.

*Memoria Técnica y Documentación: Explicación clara del paso a paso, justificación de las decisiones técnicas tomadas: explicado en en el apartado a) .

*Inclusión de capturas de pantalla del desarrollo y transparencia en el uso de la IA.

-Cabecera de usuario no logeado:



headerhome.php U

C: > xampp > htdocs > webcurso > headerhome.php > ...

```php
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

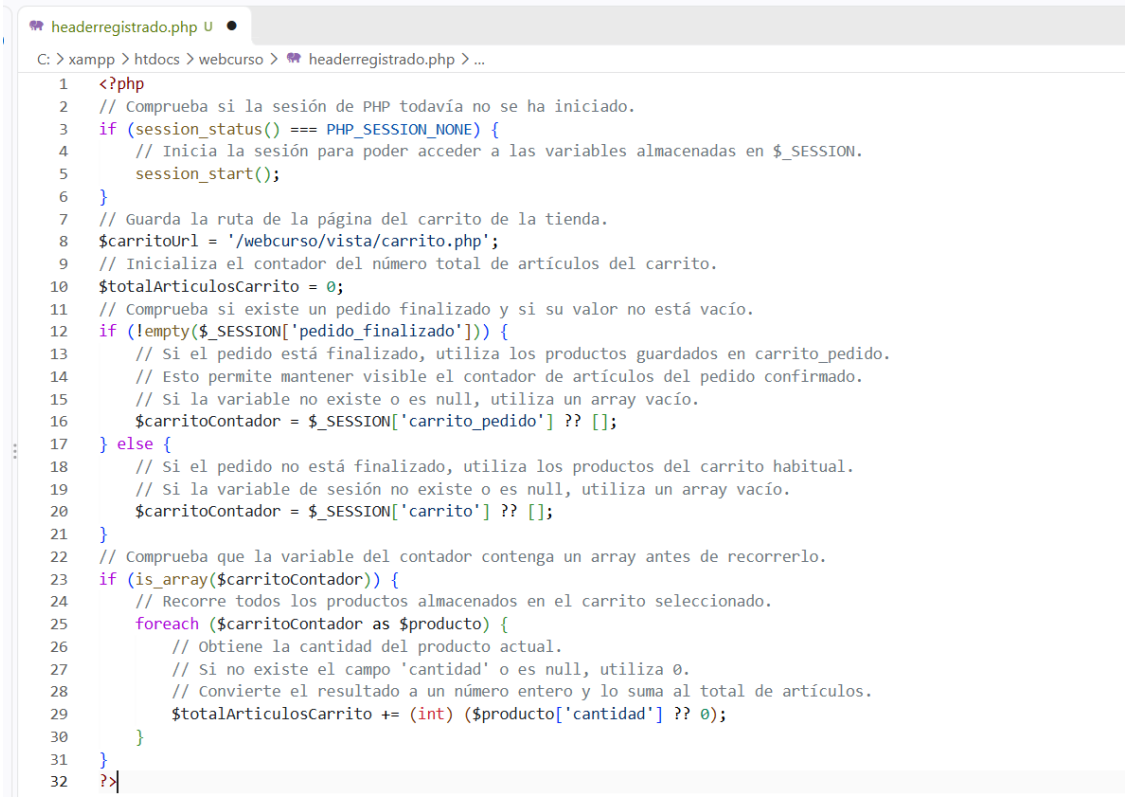
$totalArticulos = 0;

if (!empty($_SESSION['pedido_finalizado'])) {
    $carritoContador = $_SESSION['carrito_pedido'] ?? [];
} else {
    $carritoContador = $_SESSION['carrito'] ?? [];
}

if (is_array($carritoContador)) {
    foreach ($carritoContador as $producto) {
        $totalArticulos += (int) ($producto['cantidad'] ?? 0);
    }
}
?>

```

-Cabecera de usuario logeado:

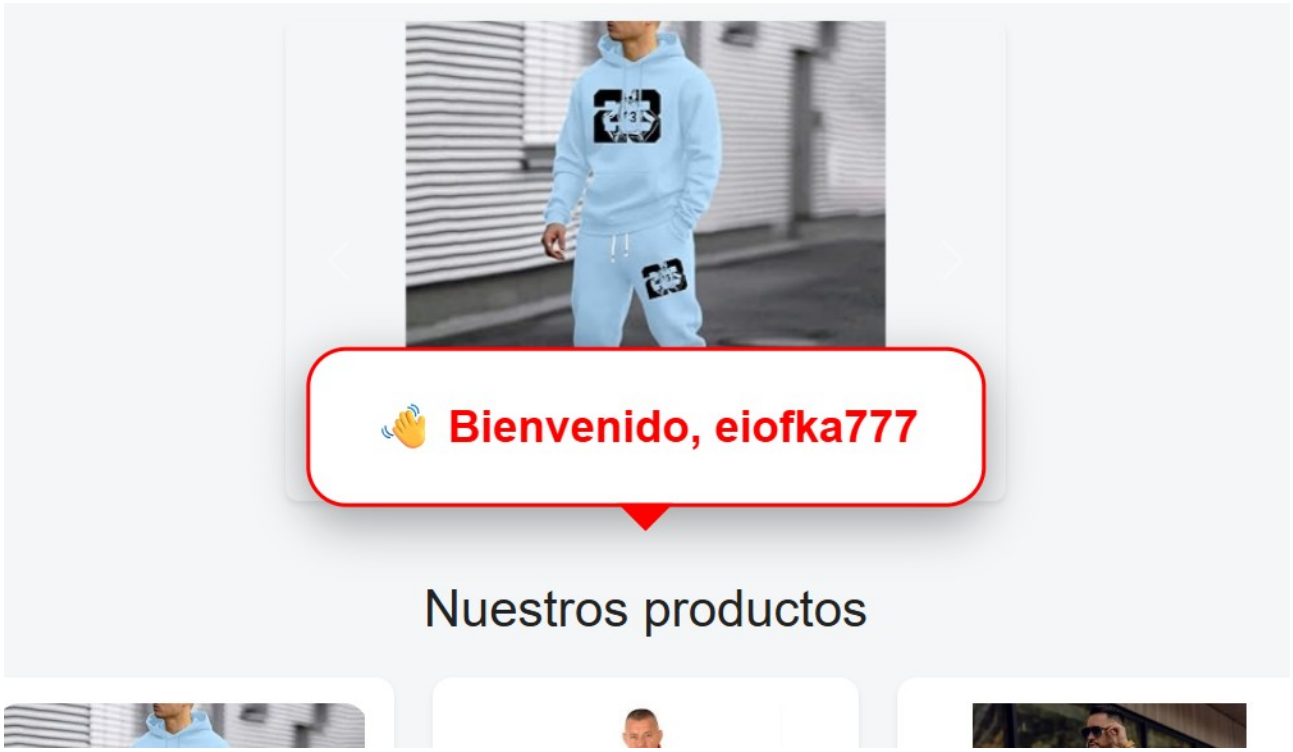


headerregistrado.php U

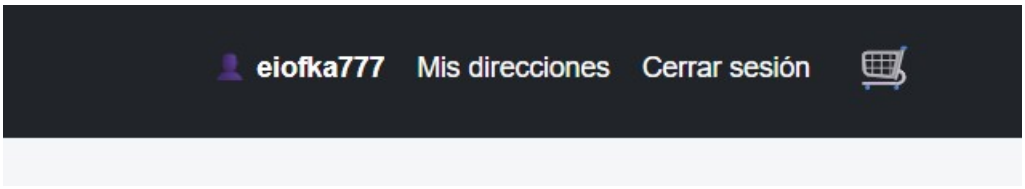
C: > xampp > htdocs > webcurso > headerregistrado.php > ...

```php
<?php
// Comprueba si la sesión de PHP todavía no se ha iniciado.
if (session_status() === PHP_SESSION_NONE) {
    // Inicia la sesión para poder acceder a las variables almacenadas en $_SESSION.
    session_start();
}
// Guarda la ruta de la página del carrito de la tienda.
$carritoUrl = '/webcurso/vista/carrito.php';
// Inicializa el contador del número total de artículos del carrito.
$totalArticulosCarrito = 0;
// Comprueba si existe un pedido finalizado y si su valor no está vacío.
if (!empty($_SESSION['pedido_finalizado'])) {
    // Si el pedido está finalizado, utiliza los productos guardados en carrito_pedido.
    // Esto permite mantener visible el contador de artículos del pedido confirmado.
    // Si la variable no existe o es null, utiliza un array vacío.
    $carritoContador = $_SESSION['carrito_pedido'] ?? [];
} else {
    // Si el pedido no está finalizado, utiliza los productos del carrito habitual.
    // Si la variable de sesión no existe o es null, utiliza un array vacío.
    $carritoContador = $_SESSION['carrito'] ?? [];
}
// Comprueba que la variable del contador contenga un array antes de recorrerlo.
if (is_array($carritoContador)) {
    // Recorre todos los productos almacenados en el carrito seleccionado.
    foreach ($carritoContador as $producto) {
        // Obtiene la cantidad del producto actual.
        // Si no existe el campo 'cantidad' o es null, utiliza 0.
        // Convierte el resultado a un número entero y lo suma al total de artículos.
        $totalArticulosCarrito += (int) ($producto['cantidad'] ?? 0);
    }
}
?>
```

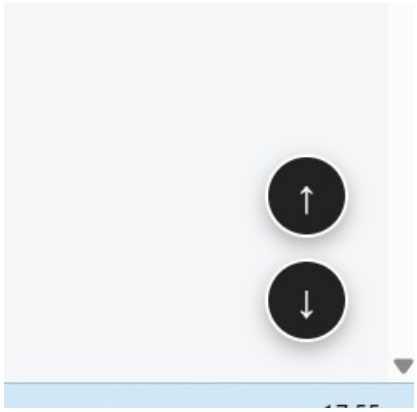
- Nombre de usuario implementado (confirmación):



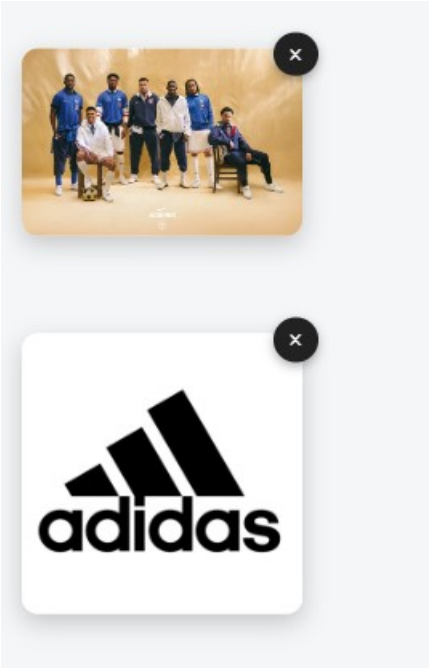
-Ya logeado:



- Comprobar funcionalidad de flechas de navegación:



- Observar funcionamiento de banners:



- Situación inicial del registro de usuario= sin sesión PHP:

registro.php U

C: > xampp > htdocs > webcurso > registro.php > ...

```php
<?php
// Inicia la sesión PHP si todavía no existe una sesión activa.
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
```

- Uso de la IA y transparencia en el uso: los archivos .odt son obtenidos del uso de IA, donde aparecen diversos comandos para optimizar el desarrollo.
